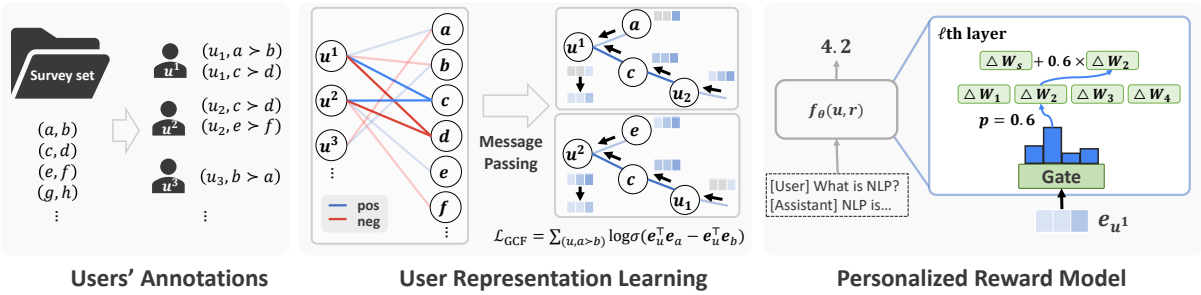

Figure 2: An overview of CoPL. To learn user representations, the GCF model is trained on a user-response bipartite graph. To build a personalized reward model, CoPL uses the learned representations to select a user-specific expert from MoLE, enabling effective modeling of diverse preferences.

After L propagation steps, user and response embeddings accumulate information from their local neighborhood. Given the final user embedding $\boldsymbol{e}_u^{(L)}$ and response embedding $\boldsymbol{e}_r^{(L)}$, we use the inner product between the embeddings as a predicted preference :

$$s_{u,r} = \left(\boldsymbol{e}_u^{(L)}\right)^\top \left(\boldsymbol{e}_r^{(L)}\right). \tag{4}$$

With the score function, the GNN is trained on preference data $\mathcal{D}_u$ for all users by minimizing the following loss function:

$$\mathcal{L}_{\text{GCF}}(\theta) := \sum_{u \in \mathcal{U}} \sum_{(a \succ b) \in \mathcal{D}_u} -\log \sigma\left(s_{u,a} - s_{u,b}\right) + \lambda \|\theta\|_2^2, \tag{5}$$

where $\sigma(\cdot)$ denotes a sigmoid function, λ is a regularization hyper-parameter and θ represents all trainable parameters, including weights of the propagation layers and initial embeddings of the users $\boldsymbol{e}_u^{(0)}$ and responses $\boldsymbol{e}_r^{(0)}$.

4.2 Personalized Reward Model with User Representations

Based on the learned user embeddings $\boldsymbol{e}_u^{(L)}$, we build a reward model that can accommodate the preferences of diverse users. We use an LLM-based reward function:

$$f_\phi(\boldsymbol{e}_u, r) : \mathbb{R}^d \times \mathcal{X} \to \mathbb{R} \tag{6}$$

where f is an LLM parameterized by ϕ taking user embedding $\boldsymbol{e}_u$ and the response r as inputs and predicts preference score. Unlike the response, the user embedding is not used as an input token. Instead, it is used in the gating mechanism described below. To learn the reward model, we can employ the BTL model, resulting in the maximum likelihood objective:

$$\mathcal{L}_{\text{RM}}(\phi) = \sum_u \sum_{(a \succ b) \in \mathcal{D}_u} \log p_\phi(a \succ b \mid \boldsymbol{e}_u) \tag{7}$$

However, naively optimizing this objective starting from a pretrained LLM requires fine-tuning billions of parameters. Moreover, different preferences of users result in conflicting descent directions of the model parameters, resembling a multi-task learning scenario.

Mixture of LoRA experts for personalized reward function. For an efficient parameter update while minimizing the negative effect of diverse preferences, we adopt the mixture of LoRA experts (MoLE) (Hu et al., 2021; Liu et al., 2024) into our framework. MoLE is proposed to maximize the benefit of the mixture of experts (MoE) while maintaining efficient parameter updates. With MoLE, the model parameter matrix W is decomposed into pretrained and frozen W_0 and trainable ΔW, i.e., $W = W_0 + \Delta W$. ΔW is further decomposed into a shared LoRA expert $A_s \in \mathbb{R}^{d_{\text{out}} \times n}, B_s \in \mathbb{R}^{n \times d_{\text{in}}}$, which is used across all users, and M individual LoRA experts $\{A_i, B_i\}_{i=1}^M$ with the same dimensionality of the shared expert. Formally, this can be written as

$$\Delta W_u = A_s B_s + \sum_{i=1}^{M} w_i A_i B_i, \tag{8}$$

where $w_i \in [0, 1]$ denotes the importance of expert i.

To adopt the different preferences of users, we define a user-dependent gating mechanism to model the importance parameter w_i. For each user u, a gating function $g : \mathbb{R}^d \to \mathbb{R}^M$ maps $\boldsymbol{e}_u^{(L)}$ to

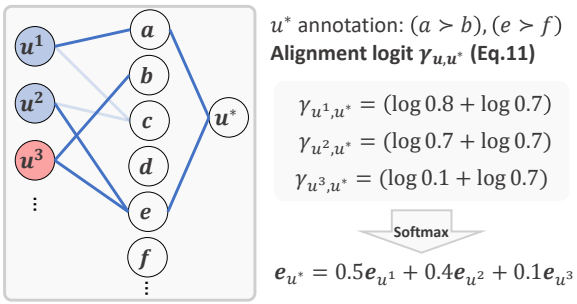

Figure 3: Illustration of unseen user adaptation. Blue nodes are users who have similar preferences to u^*, and red nodes are users who have dissimilar preferences.

expert-selection logits:

$$\mathbf{z} = g\big(\boldsymbol{e}_u^{(L)}\big). \tag{9}$$

We convert these logits $\mathbf{z}$ into gating weight w_i by selecting the top one expert from the logits:

$$w_i = \begin{cases} \frac{\exp(z_i/\tau)}{\sum_{j=1}^{M}\exp(z_j/\tau)} & \text{if } i = \arg\max_i z_i \\ 0 & \text{otherwise,} \end{cases} \tag{10}$$

where τ is a temperature parameter. In practice, one can use top-k experts, but we could not find a significant difference in our experiments. For computational efficiency, we keep the top one expert.

4.3 Optimization-free User Adaptation

While we can predict a preference score of unseen responses for a known user, the reward model trained in Section 4.2 cannot be used to predict the preference of users who have not been observed during training. To estimate the embeddings of unseen users, we propose an optimization-free adaptation approach.

Let u^* be an unseen user who annotates a small set of response pairs. Under the assumption that users who have similar responses have similar preferences, we can estimate the embedding of an unseen user by taking an embedding of users with similar tastes. For example, if both user u^* and u share positive preference over the same response r, then we can use the embedding of u to approximate that of u^*. Based on this intuition, we propose the following optimization-free adaptation strategy for unseen user embedding:

$$\boldsymbol{e}_{u^*}^{(L)} = \sum_{u \in \mathcal{N}_{u^*}^{+}(k)} w_{u,u^*}\boldsymbol{e}_u^{(L)}, \tag{11}$$

Dataset	TL;DR	UF-P-2	UF-P-4	PersonalLLM
Size of survey set	19,824	25,993	25,993	14,435
# of preference groups	2	2	4	∞
# of annotations per user	8	8	16	16
# of users per group	5,000	5,000	2,500	-

Table 1: Statistics of the datasets. We report the *average* number of annotations per user. All users have different preferences in PersonalLLM.

where $\mathcal{N}_{u^*}^{+}(k)$ is a set of k-hop neighborhood[1] of user u^* connected by only positive edges, and w_{u,u^*} is a normalized alignment score between u and u^*. The normalized alignment score w_{u,u^*} is defined as

$$w_{u,u^*} = \frac{\exp(\gamma_{u,u^*}/\kappa)}{\sum_{\tilde{u} \in \mathcal{N}_{u^*}^{+}(k)} \exp(\gamma_{\tilde{u},u^*}/\kappa)},$$

where

$$\gamma_{u,u^*} = \sum_{(a \succ b) \in \mathcal{D}_{u^*}} \log \sigma(s_{u,a} - s_{u,b}),$$

where $s_{u,i}$ is an inner product between user and response embeddings, κ is a temperature parameter, and γ_{u,u^*} is an alignment score between user u and u^*. Intuitively, γ_{u,u^*} measures how well the *predicted preference* of user u aligns with the *annotated preference* provided by user u^*. If the preferences of both users align well, γ_{u,u^*} is large. Consequently, their embeddings become similar to each other. By collecting embeddings of well-aligned neighborhood users, we can obtain embeddings of user u^* without having further optimization.

5 Experiments

In this section, we empirically verify the performance of CoPL across various scenarios.

5.1 Experimental Settings

Datasets. We employ three datasets, including TL;DR (Stiennon et al., 2020; Chen et al., 2024a), UltraFeedback-P (UF-P) (Poddar et al., 2024), and PersonalLLM (Zollo et al., 2024), that explicitly capture diverse user preferences rather than assuming a single dominant preference. We briefly describe the key characteristics of these datasets below.

Following prior work (Chen et al., 2024a; Li et al., 2024), we define two user groups in the TL;DR dataset: one group prefers short summaries,

[1] k must be an even number to aggregate only the user embeddings.

and the other favors long summaries. We create two environments with the UF-P dataset: UF-P-2, dividing users into two groups based on their preference, and UF-P-4, dividing users into four groups. In PersonalLLM (Zollo et al., 2024), user preferences are modeled as a mixture of four preference dimensions where weight vectors are drawn from a Dirichlet distribution with $\alpha = 0.1$. Additional details on their construction and properties can be found in Section D.1.

We divide $10,000$ users evenly into the predefined number of preference groups. For all datasets, we curate two different versions, denoted as ALL and AVG, representing two different annotation sampling strategies. For TL;DR/UF-P-2 (ALL), each user provides exactly 8 annotations, while for TL;DR/UF-P-2 (AVG), each user's annotation count is uniformly sampled from 1 to 15, averaging to 8. Similarly, in UF-P-4/PersonalLLM (ALL), each user provides exactly 16 annotations, and in UF-P-4/PersonalLLM (AVG), the count is uniformly sampled from 1 to 31, averaging to 16. Table 1 summarizes the key statistics.

Baselines. We evaluate six baselines to benchmark. First, we use a uniform preference model (Uniform) trained on all annotations via BTL. Additionally, we consider four personalized reward models: I2E, $\text{I2E}_{\text{proxy}}$ (Li et al., 2024), VPL (Poddar et al., 2024), and PAL (Chen et al., 2024a). Finally, we include a group-wise Oracle (G-Oracle), which has access to user group information and all annotations in the survey set, and trains a separate reward function in Eq. (1) for each preference group. Note that we do not have the G-Oracle for PersonalLLM since the users are not categorized into a fixed number of preference groups. The details of each model are provided in Section B.

Training and evaluation details. For reward function training, we utilize two LLM backbones: `gemma-2b-it` and `gemma-7b-it` (Team et al., 2024a). Our model uses one shared LoRA, eight LoRA experts, each with a rank of eight, and a two-layer MLP for the gating function. The other baselines, e.g., Uniform, I2E, VPL, PAL, and G-Oracle, use a LoRA rank of 64. Other training details, such as hyper-parameters and model architecture, are provided in Section D.2. All experiments, including additional analysis, are repeated three times with different seeds.

We report reward model accuracy on unseen test pairs that are not in the survey set. We evaluate performance for both seen and unseen users. For seen user experiments, each user is assigned 10 test pairs, and accuracy is calculated over all seen users. We fix the number of unseen users at 100, evenly distributed across preference groups. To adapt the reward model for each unseen user, we provide 8 annotations in TL;DR/UF-P-2 (ALL/AVG) and 16 annotations in UF-P-4/PersonalLLM (ALL/AVG), followed by evaluation on 50 test pairs per unseen user. CoPL uses 2-hop neighbors for unseen user adaptation.

5.2 Results

Table 2 presents accuracy for both seen and unseen users. CoPL consistently outperforms other baselines, except for G-Oracle, in both seen user and unseen user experiments. Notably, CoPL surpasses the performance of G-Oracle on TL;DR and UF-P-4, demonstrating the advantage of multi-task learning. In the PersonalLLM, CoPL remains robust across the ALL and AVG, whereas VPL suffers from performance degradation in a more realistic AVG setting. These findings are consistent with Ju et al. (2024), which theoretically shows that message-passing can help users with limited interactions in collaborative filtering. In unseen user experiments, CoPL achieves accuracy comparable to the seen user setting, indicating the effectiveness of our unseen user adaptation.

Fig. 1 illustrates the learned user embeddings for UF-P-4 (AVG), selected as the most challenging environment among those with distinct groups. The figure shows that GNN-based representation learning successfully captures preference similarities, despite the limited annotations per user.

5.3 Analysis

Analysis of performance in UF-P-2. In Table 2, all models appear capable of representing diverse preferences, surprisingly including the uniform models in UF-P-2 (ALL/AVG). To investigate further, we divide the test pairs of UF-P-2 into *common* and *controversial* categories, where common pairs have identical annotations from both preference groups, and controversial pairs differ. Focusing on the seen user results in UF-P-2 (ALL) with `gemma-2b-it` from Table 2, we break down the accuracy in Table 3. The results indicate that baselines, except G-Oracle, struggle with controversial pairs, suggesting a tendency to capture only the common preference across all users. By contrast, our method achieves comparable performance to